MCA Data Analytics Lab Test 1

Complete workflow based on the uploaded lab manual.

Run the cells sequentially.

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, MinMaxScaler

np.random.seed(42)
```

1. Synthetic Dataset Creation

Paste the complete synthetic dataset generation code from the lab manual here (Academic_Performance.csv and Placement.csv generation).

```
# Paste the complete synthetic dataset generation code from the lab
manual here.
# It generates:
# - Academic_Performance.csv
# - Placement.csv
# and saves them to Google Drive.
```

2. Handle Missing Values & Duplicates

```
numeric_columns = academic.select_dtypes(include=np.number).columns
for col in numeric_columns:
    academic[col].fillna(academic[col].mean(), inplace=True)

categorical_columns = academic.select_dtypes(include='object').columns
for col in categorical_columns:
    academic[col].fillna(academic[col].mode()[0], inplace=True)

placement["Package_LPA"].fillna(placement["Package_LPA"].median(),
inplace=True)
placement["Company_Name"].fillna("Not Available", inplace=True)

print("Academic duplicates:", academic.duplicated().sum())
print("Placement duplicates:", placement.duplicated().sum())
```

```
academic = academic.drop_duplicates()
placement = placement.drop_duplicates()
```

3. Detect & Remove Outliers

```
plt.figure(figsize=(10,6))
sns.boxplot(data=academic[["Attendance","Study_Hours","Percentage"]])
plt.show()

def remove_outliers(df,column):
    Q1=df[column].quantile(0.25)
    Q3=df[column].quantile(0.75)
    IQR=Q3-Q1
    lower=Q1-1.5*IQR
    upper=Q3+1.5*IQR
    return df[(df[column]>=lower)&(df[column]<=upper)]

academic=remove_outliers(academic,"Percentage")
academic=remove_outliers(academic,"Attendance")
```

4. Feature Engineering

```
academic["Average_Score"]=(academic["Assignment_Marks"]
+academic["Internal_Marks"]+academic["Final_Marks"])/3

academic["Attendance_Category"]=np.where(
    academic["Attendance"]>=85,
    "High",
    np.where(academic["Attendance"]>=70,"Medium","Low")
)

conditions=[
    academic["Percentage"]>=85,
    academic["Percentage"]>=70,
    academic["Percentage"]>=55
]
choices=["Excellent","Good","Average"]

academic["Performance"]=np.select(conditions,choices,default="Poor")

encoder=LabelEncoder()
for col in ["Gender","Department","City","Result","Internet_Access",

"Extra_Curricular","Attendance_Category","Performance","Grade"]:
    academic[col]=encoder.fit_transform(academic[col])

scaler=MinMaxScaler()
cols=["Attendance","Study_Hours","Assignment_Marks","Internal_Marks",
```

```
"Final_Marks","Average_Marks","Average_Score","Percentage","Family_Income"]
academic[cols]=scaler.fit_transform(academic[cols])
```

5. Merge, Save & Reload

```
academic["Assessment_Date"]=pd.to_datetime(academic["Assessment_Date"])

merged=pd.merge(academic,placement,on="Student_ID",how="left")

merged.to_csv(
"/content/drive/My Drive/Lab_BigData/Academic_Performance_Preprocessed.csv",
index=False
)

final_df=pd.read_csv(
"/content/drive/My Drive/Lab_BigData/Academic_Performance_Preprocessed.csv"
)

print(final_df.head())
print(final_df.shape)
```